

Q1

$$a) \quad T(n) = T(n-1) + c$$

$$T(n-1) = T(n-2) + c$$

$$T(n-2) = T(n-3) + c$$

$$\vdots$$

$$T(1) = T(0) + c \rightarrow T(0) = 1 \rightarrow \text{base case}$$

$$T(1) = c$$

$$+ \quad \frac{n-c}{n-c} \implies T(n) \in \Theta(n)$$

$$b) \quad \text{We can derive its formula: } T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

$$T(n) = 2 T\left(\frac{n}{2}\right) + c$$

 $a \rightarrow \text{sub-problem}$ 
 $\frac{n}{b} \rightarrow \text{size of the sub-problem}$ 

Master Theorem

$$a=2$$

$$b=2$$

$$c = \Theta(1) \rightarrow \text{because constant}$$

$$\log_b^a = \log_2^2 = 1 \rightarrow T(n) = \Theta(n^{\log_b^a}) = \Theta(n^1) = \underline{\underline{\Theta(n)}}$$

Since both of the algorithms have the same time-complexity, it would not matter which algorithm you should use.

But, algorithm 2 calls itself 2 times, and algorithm 1 calls itself only once. So, it would be maybe better to use algorithm 1, space wise.

Q2

```
def Polynomial(int x, array):  
    int result = 0  
    for i in range(array.length):  $\rightarrow$  exponent  
        result = result + (x**i)*array[i]  
    return result
```

The inside of the for loop is a basic operation, and its time complexity is  $\Theta(1)$

There is only one loop, which runs  $n$  times. Thus, the time complexity is  $\rightarrow \Theta(n)$

A better way?

$\rightarrow$  There is no better way to implement this algorithm with a brute-force style.  
It is already as fast as possible.

Q3

```
def CountSubstring(str, start, end):  
    int count = 0  
    for i in range(str.length):  
        if (str[i] == start):  
            for j in range(i, str.length):  
                if (str[j] == end):  
                    count = count + 1  
    return count
```

$\rightarrow$  As we can see, there are 2 for loops  
in which they consist  $O(1)$  basic operations.

In the worst-case:

$$\sum_{i=1}^n \sum_{j=1}^n 1 = \sum_{i=1}^n (n-1) = \frac{n \cdot (n-1)}{2} \approx n^2$$

$$T_{\text{worst}} = \Theta(n^2)$$

Best case

$\rightarrow$  it doesn't go to second loop

$$\sum_{i=1}^n 1 = n \Rightarrow \Theta(n)$$

$$T_{\text{best}} = \Theta(n)$$

Q4

def Distance (p1, p2) :

// question assumes that this is given.

def BruteForce ClosestDistance ( P, n )  $\xrightarrow{\text{Points array}}$   $\xrightarrow{\text{size}}$

min\_value = float('inf') //  $\rightarrow$  means infinity ( $\infty$ )

for i in range (n) :

for j in range (i+1, n) :

if ( Distance (P[i], P[j]) < min\_value ) :

min\_value = Distance (P[i], P[j])

return min\_value

~~~~~

First loop runs n times, second loop runs (n-i) times.

So, total number of iterations = (n-1) + (n-2) + (n-3) + ... + 2 + 1

$$= \frac{n \cdot (n-1)}{2} \approx n^2 \rightarrow \Theta(n^2)$$

Thus, time complexity of this algorithm is  $\Theta(n^2)$

$$\sum_{i=1}^n \sum_{j=i+1}^n 1 = \sum_{i=1}^n (n-i) \\ = \frac{n \cdot (n-1)}{2}$$

Q5

a) def Profitable Cluster (regions, profits):

maxProfit = profits[0]

cluster = [regions[0]]

for i in range (len (regions)):

for j in range (i, len (regions)):

p = sum (profits, i, j)

if (maxProfit < p):

maxProfit = p

cluster = regions [i:j]

return cluster

def sum (values, start, end):

result = 0

for i in range (start, end):

result = result + values[i]

return result

We can see the function has 2 loops, which runs essentially  $n$  times. But in the second for loop we are calling the sum function, which is  $\Theta(n)$  time. This makes it so it is actually  $\Theta(n^3)$  time.

b) Since this is a recursive method, we can derive its recurrence relation:

$$T(n) = 2 T\left(\frac{n}{2}\right) + \Theta(n)$$

Master method

$$a = 2$$

$$b = 2$$

$$c = \Theta(n)$$

$$\log_b a = 1$$

$$c = \Theta(n)$$

case 2

$\Rightarrow$

$$T(n) = \Theta(n \cdot \log n)$$